Vision 기본 모델 Day 1

NumPy 에서 PyTorch 로, 그리고 영상 데이터

HYUNDAI AI Insight Campus · 온디바이스 AI · Vision 기본 모델 1/5

2026. 10. 27 (화) 09:00–18:00 · 8교시 · Google Colab (하드웨어 없음)

Tensor | autograd | nn.Module | DataLoader | 이미지 | 영상

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 PyTorch 전환

mynn.py 의 계층 · 학습 코드를 PyTorch 부품으로 옮기고 SimpleConvNet 433,890 파라미터를 확인한다

2 Tensor · GPU

from_numpy(공유) 와 tensor(복사), float32, permute · unsqueeze, .to(device) 를 구분해 쓴다

3 autograd

backward() 로 기울기를 구하고 손 유도 $dW = X^T(Y - t)/N$ 과 같은지 확인한다

4 nn.Module · DataLoader

모델 클래스 · Dataset · DataLoader 로 학습 루프를 만들고 평가 · 저장한다

5 이미지 · 영상

OpenCV (H, W, C) BGR 을 (C, H, W) RGB 텐서로 바꾸고 VideoCapture → VideoWriter 루프를 만든다

6 미니 프로젝트

종이 손글씨 사진 30 장과 영상 속 숫자를 전처리해 인식한다 (naive 0.100 → 약 0.967)

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 PyTorch 부품 , 오후 학습 루프 → 이미지 → 영상 → 미니 프로젝트

교시	시간	주제	형태	애니메이션
1	09:00–09:50	NN 복습 퀴즈 · mynn → PyTorch	퀴즈 + 강의	D1-1 mynn → PyTorch
2	10:00–10:50	Tensor · GPU	강의 + 실습	D1-2 Tensor 의 메모리
3	11:00–11:50	autograd — 역전파를 자동으로	강의 + 실습	D1-3 autograd 계산 그래프
4	13:00–13:50	nn.Module · Dataset · DataLoader	강의 + 실습	D1-4 Dataset · DataLoader
5	14:00–14:50	학습 루프 · 평가 · 저장	강의 + 실습	D1-5 학습 루프 한 바퀴
6	15:00–15:50	디지털 이미지 — 픽셀 · 채널 · 정규화	강의 + 실습	D1-6 이미지 → 텐서
7	16:00–16:50	영상 파일 파이프라인	강의 + 실습	D1-7 영상 = 프레임의 연속
8	17:00–17:50	미니 프로젝트 — 종이 손글씨 · 영상 숫자 인식	개인 실습	D1-8 종이 손글씨 전처리

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 1)

https://imguru-mooc.github.io/AI_CAMPUS/VB-day1/

animation.html	애니메이션 8 개 (D1-1 ~ D1-8) — #d1-6 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
VB_Day1_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab 에서 열기)
VB_Day1.pdf	오늘 슬라이드 PDF
data/paper_digits.zip	종이 손글씨 사진 30 장 (숫자 0~9 × 3 장) — 6 · 8 교시 실습
NN Day 5 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 런타임 유형은 GPU · 하드웨어 없이 데이터만으로 진행합니다

오늘의 큰 그림 — 같은 모델 , 다른 도구

NN 과정에서 NumPy 로 원리를 만들었습니다 . 오늘은 같은 모델을 PyTorch 로 옮기고 , 실제 사진 · 영상을 넣어 봅니다

NN 과정 (NumPy · 원리)

```
mynn.py — 계층을 직접 구현
im2col · backward 직접 작성
SimpleConvNet 1 epoch 0.9810
```

params **433,890**

오늘 바꾸는 것

계산	torch.Tensor · GPU
미분	autograd
부품	nn · optim · DataLoader
입력	OpenCV 이미지 · 영상

Vision 과정 (PyTorch · 실제 데이터)

```
nn.Conv2d · MaxPool2d · Linear
loss.backward() · opt.step()
종이 사진 · 영상 프레임 입력
```

params **433,890**

Vision 기본 모델 5 일 로드맵

Day 1 10/27(화)
PyTorch · 이미지 · 영상 → 종이 손글씨

→

Day 2 10/28(수)
LeNet · VGG · ResNet → CIFAR-10

→

Day 3 10/29(목)
경량 CNN · 전이학습 · Grad-CAM

→

Day 4 11/2(월)
객체 탐지 IoU · NMS · mAP · YOLO

→

Day 5 11/3(화)
세그멘테이션 · 포즈 · ONNX → 침입 감지

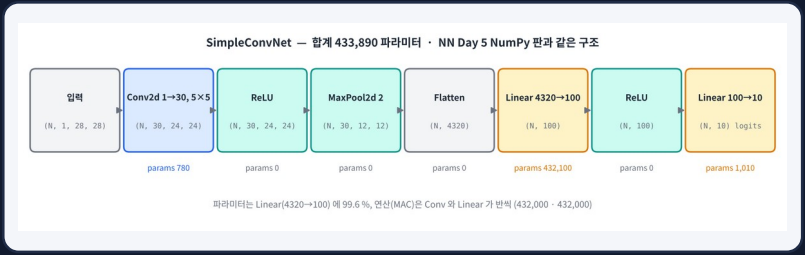
후속 : ONNX · TensorRT (11/4~) → Quantization · Pruning → [프로젝트] 현장 안전 모니터링

1 NN 복습 퀴즈 · mynn → PyTorch

09:00–09:50

학습 목표

- 09:00–09:15 NN Day 5 복습 퀴즈 (20 문항)
- mynn.py 부품 ↔ PyTorch 부품 대응표
- SimpleConvNet 을 nn.Sequential 로 다시 쓰기
- 파라미터 433,890 — NumPy 판과 같음



노트북 실습

1. 문제 1-1 nn.Sequential + 파라미터 수
2. 문제 1-2 계층별 shape 추적 (x = layer(x))

▶ 애니메이션 D1-1 mynn → PyTorch

핵심 개념

대응표

```
Affine → nn.Linear   (W 는 (out, in), x @ W.T + b)
SoftmaxWithLoss → nn.CrossEntropyLoss (logits)
optimizer.update → zero_grad → backward → step
```

모델

```
nn.Sequential(Conv2d(1,30,5), ReLU, MaxPool2d(2),
  Flatten, Linear(4320,100), ReLU, Linear(100,10))
```

shape

```
(2,1,28,28) → Conv (2,30,24,24) → Pool (2,30,12,12)
→ Flatten (2,4320) → Linear (2,100) → Linear (2,10)
```

파라미터

```
Conv 30·1·5·5+30 = 780   Linear1 = 432,100
Linear2 100·10+10 = 1,010   합계 433,890
```

세기

```
sum(p.numel() for p in net.parameters())
```

체크포인트

구조 · 파라미터 수는 NumPy 판과 똑같다. 달라지는 것은 im2col 과 backward 를 직접 짜지 않는다는 것 — Conv2d 내부에서 처리하고 GPU 에서는 cuDNN 이 계산한다.

2 Tensor · GPU

10:00–10:50

학습 목표

- Tensor = ndarray + device + requires_grad
- 기본 float32 (NumPy 기본 float64)
- from_numpy 는 공유 , torch.tensor 는 복사
- reshape · permute · unsqueeze
- .to(device) — 모델과 데이터는 같은 device

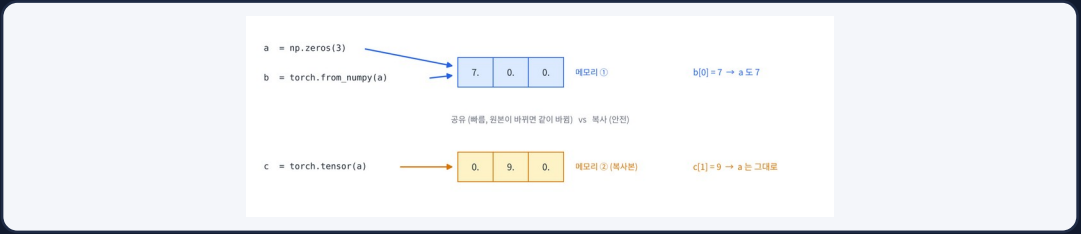
노트북 실습

1. 문제 2-1 dtype 함정 (float64 → .float())
2. 문제 2-2 공유 vs 복사 예측
3. 문제 2-3 Y = XW + b · softmax 를 torch 로
4. 문제 2-4 permute · unsqueeze · reshape

▶ 애니메이션 D1-2 Tensor 의 메모리

핵심 개념

공유 vs 복사



문제 2-2

```
b = torch.from_numpy(a);  c = torch.tensor(a)
b[0] = 7;  c[1] = 9   →  a = [7, 0, 0],  c = [0, 9, 0]
```

softmax

```
Y = X @ W + b;  Y = Y - Y.max(dim=1, keepdim=True).values
E = torch.exp(Y);  P = E / E.sum(dim=1, keepdim=True)
```

shape

```
x = torch.arange(24).reshape(2, 3, 4)
permute(0, 2, 1) → (2, 4, 3)   unsqueeze(0) → (1, 2, 3, 4)
reshape(2, -1) → (2, 12)
```

체크포인트

float64 텐서를 nn.Linear 에 넣으면 expected Float but found Double → .float(). GPU 시간은 torch.cuda.synchronize() 후 측정 (2048×2048 행렬곱 CPU 약 77.5 ms, 실행 예).

3 autograd — 역전파를 자동으로

11:00–11:50

학습 목표

- requires_grad=True → 연산을 계산 그래프로 기록
- backward() 가 연쇄 법칙을 거꾸로 적용
- 잎 텐서 .grad 에 누적 (더해짐)
- 손 유도 기울기와 autograd 비교

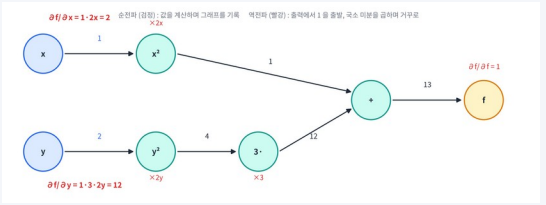
노트북 실습

1. 문제 3-1 f = x² + 3y² 미분
2. 문제 3-2 기울기 누적 · zero_()
3. 문제 3-3 Affine + softmax + CE 비교
4. 문제 3-4 autograd 경사하강
5. 문제 3-5 torch.optim.SGD

▶ 애니메이션 D1-3 autograd 계산 그래프

핵심 개념

계산 그래프



3-1 · 3-2

```
f = x² + 3y² at (1, 2) → f = 13, ∂f/∂x = 2, ∂f/∂y = 12
(x**2).backward() 두 번 → x.grad 2 → 4 ( 누적 )
```

3-3

```
dW = X.T @ (Y − T) / N      db = (Y − T).sum(0) / N
autograd 의 W.grad 와 일치 ( 최대 오차 약 3e-8)
```

3-4 · 3-5

```
with torch.no_grad(): p -= lr * p.grad;  p.grad.zero_()
opt.zero_grad() → f.backward() → opt.step()  → (0, 0) 근처
```

체크포인트

zero_grad 를 빠뜨리면 기울기가 계속 누적된다 . requires_grad 텐서는 .detach().numpy(). no_grad 없이 in-place 갱신하면 "a leaf Variable that requires grad ..." 오류 .

4 nn.Module · Dataset · DataLoader

13:00–13:50

학습 목표

- nn.Module: __init__ 에서 계층 , forward 에서 연결
- self. 에 붙인 계층 → parameters() 자동 수집
- Dataset (__len__ · __getitem__) → DataLoader
- CrossEntropyLoss = logits + 정수 레이블

노트북 실습

1. 문제 4-1 MLP 784-100-10 (79,510)
2. 문제 4-2 NumpyDigits(Dataset)
3. 문제 4-3 DataLoader 배치 100
4. 문제 4-4 CrossEntropyLoss 직접 계산

▶ 애니메이션 D1-4 Dataset · DataLoader

핵심 개념

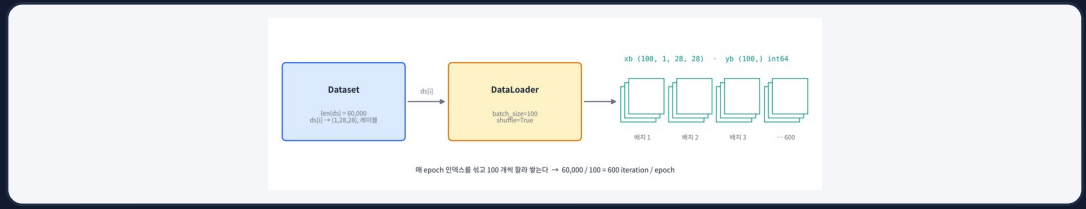
MLP

```
self.fc1 = nn.Linear(784, 100);  self.fc2 = nn.Linear(100, 10)
forward: x.flatten(1) → F.relu(self.fc1(x)) → self.fc2(x)
파라미터 79,510 (NN Day 3 과 같음 )
```

Dataset

```
MNIST train 60,000 · test 10,000 · ToTensor() → (1, 28, 28)
__getitem__: torch.from_numpy(X[i]).float() / 255
             .unsqueeze(0),  int(y)
```

DataLoader



손실

```
-F.log_softmax(logits, dim=1)[torch.arange(N), yb].mean()
학습 전 2.3010 ≈ ln 10 = 2.3026
```

체크포인트

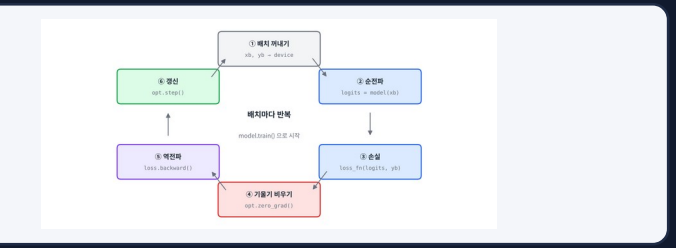
배치 xb (100, 1, 28, 28) · yb int64 (100,) · len(train_loader) = 600. 모델 끝에 softmax 를 또 붙이지 않는다 — CrossEntropyLoss 가 포함한다 .

5 학습 루프 · 평가 · 저장

14:00–14:50

학습 목표

- 배치마다 6 단계 — model.train() 으로 시작
- 평가 : model.eval() + torch.no_grad()
- state_dict 로 저장 · 불러오기



노트북 실습

1. 문제 5-1 train_one_epoch
2. 문제 5-2 evaluate
3. 문제 5-3 MLP · Adam 3 epoch
4. 문제 5-4 저장 · 불러오기

▶ 애니메이션 D1-5 학습 루프 한 바퀴

핵심 개념

6 단계

```
① xb, yb → device    ② logits = model(xb)
③ loss = loss_fn(logits, yb)    ④ opt.zero_grad()
⑤ loss.backward()    ⑥ opt.step()
```

evaluate

```
model.eval()
with torch.no_grad(): pred = model(xb).argmax(dim=1)
학습 전 정확도 0.0953
```

3 epoch (실행 예)

```
epoch 1   loss 0.4043   test 0.9356   4.9 s
epoch 2   loss 0.1891   test 0.9568   4.9 s
epoch 3   loss 0.1372   test 0.9624   4.4 s
```

저장

```
torch.save(model.state_dict(), "mlp.pt")   # 320,309 bytes
new.load_state_dict(torch.load("mlp.pt", map_location=device))
```

체크포인트

eval() 은 Dropout · BN 모드 , no_grad() 는 기록 끄기 — 다른 일 . model.to(device) 는 옵티마이저 생성 전에 , 손실은 loss.item() 으로 모은다 . (NN Day 3: SGD 10 epoch 0.9694)

6 디지털 이미지 — 픽셀 · 채널 · 정규화

15:00–15:50

학습 목표

- OpenCV (H, W, C) uint8 BGR
- PyTorch (C, H, W) float32 0~1 RGB
- Gray = 0.299R + 0.587G + 0.114B
- 학습과 추론의 전처리는 같아야 한다

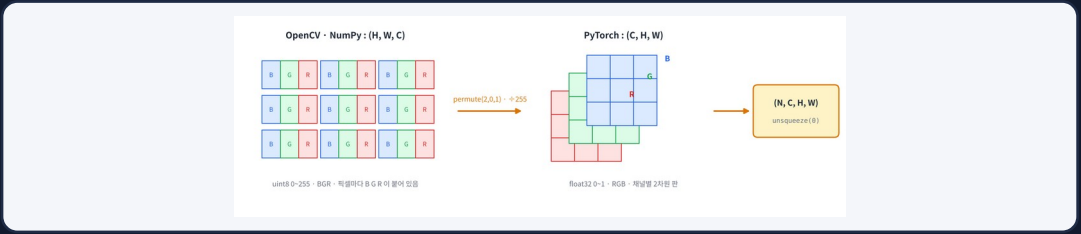
노트북 실습

1. 문제 6-1 BGR → RGB
2. 문제 6-2 흑백 직접 계산
3. 문제 6-3 ToTensor 직접
4. 문제 6-4 crop · resize 224
5. 문제 6-5 ImageNet 정규화

▶ 애니메이션 D1-6 이미지 → 텐서

핵심 개념

HWC → CHW



읽기

```
img = cv2.imread("data/paper_digits/d3_0.jpg")
# (640, 480, 3) uint8 BGR · 값 25~186 → rgb = img[:, :, ::-1]
```

overflow

```
uint8 [[200, 250]] + 100 → [[44, 94]]   float → [300, 350]
cv2.add → [[255, 255]]  (saturation)
```

텐서로

```
torch.from_numpy(rgb.copy()).permute(2, 0, 1).float() / 255
cv2.resize(crop, (224, 224), interpolation=cv2.INTER_AREA)
```

체크포인트

cv2.resize 의 크기 인자는 (W, H). 정규화 mean [0.485, 0.456, 0.406] · std [0.229, 0.224, 0.225] 를 mean.view(3, 1, 1) 로 브로드캐스팅 → (1, 3, 224, 224).

7 영상 파일 파이프라인

16:00–16:50

학습 목표

- 영상 = 프레임의 연속 (FPS · 프레임 수)
- VideoCapture → 처리 → VideoWriter 루프
- 크기 인자 (W, H) — 틀리면 빈 영상

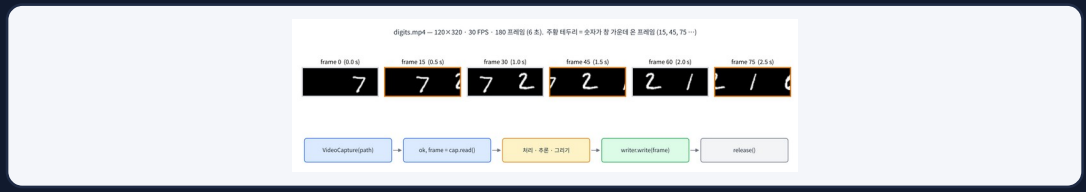
노트북 실습

1. 문제 7-1 VideoWriter 로 digits.mp4
2. 문제 7-2 영상 속성 읽기
3. 문제 7-3 frame_to_input
4. 문제 7-4 읽기 → 처리 → 쓰기 루프

▶ 애니메이션 D1-7 영상 = 프레임의 연속

핵심 개념

digits.mp4



영상 정보

```
120×320 · 30 FPS · 180 프레임 · 6.0 초 · [7, 2, 1, 0, 4, 1]
가운데 프레임 = 30k + 15 → 15, 45, 75, 105, 135, 165
```

쓰기

```
cv2.VideoWriter("digits.mp4", cv2.VideoWriter_fourcc(*"mp4v"),
                FPS, (W_V, H_V))     # 크기는 (W, H) · 약 43 KB
```

입력 변환

```
BGR2GRAY → gray[:, 100:220] → resize 28×28 INTER_AREA
→ /255 → [None, None]  →  (1, 1, 28, 28)
```

루프

```
while True: ok, frame = cap.read(); if not ok: break
→ 상자 · 프레임 번호 → write · 프레임당 평균 약 0.7 ms
```

체크포인트

cap.get(CAP_PROP_...) 은 float → int. release() 를 꼭 부르고 ok 를 확인한다. mp4v 는 브라우저 재생이 안 될 수 있어 H.264 로 변환해 재생.

8 미니 프로젝트 — 종이 손글씨 · 영상 숫자 인식

17:00–17:50

강의 노트북 8 교시 · preprocess(img_bgr) → (1, 1, 28, 28)

```
① cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
② bg = GaussianBlur(g, (0,0), 25); g = cv2.divide(g, bg, scale=255)
③ GaussianBlur (5,5) → Otsu BINARY_INV → MORPH_OPEN 3×3
④ findContours → max(contourArea) → boundingRect → 자르기
⑤ scale = 20 / max(w, h)  (긴 변 20, 비율 유지)
⑥ 28×28 가운데 → moments 무게중심 → (14, 14) 로 이동 → ÷255
```

과제	내용	배점
1	SimpleConvNet nn.Module (433,890)	20
2	2 epoch 학습 (Adam 1e-3, 배치 100) — test ≥ 0.98	25
3	preprocess(img_bgr) → (1, 1, 28, 28)	25
4	30 장 정확도 : naive vs preprocess	10
5	영상 프레임 인식 · digits_pred.mp4 · 정확도 1.0	10
6	결과 해석 (마크다운, 강사 채점)	10

채점 셀 실행 → 자동 채점 90 점 + 강사 10 점

▶ D1-8 종이 손글씨 전처리 — 단계별 변화를 먼저 보고 코드로 확인

결과 (실행 예, CPU)

과제 2	loss	test	시간
epoch 1	0.2366	0.9791	13.6 s
epoch 2	**0.0680**	**0.9830**	**14.6 s**

30 장 · 영상	정확도
naive (흑백 → 28×28 → ÷255)	0.100 (3/30)
조명 보정 없이	0.600 (18/30)
전체 preprocess	**0.967 (29/30)**
영상 가운데 프레임 [7, 2, 1, 0, 4, 1]	**1.0**



naive 는 27 장을 '8' 로 — 흰 배경이 전부 밝은 픽셀. 틀린 1 장 d7_2 → 2.
domain shift — 오늘은 입력을 학습 데이터 (MNIST) 쪽으로 옮긴다.

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

mynn → PyTorch	같은 SimpleConvNet 433,890 — im2col · backward 는 PyTorch 가 처리	D1-1
Tensor	float32 기본 · from_numpy 공유 / tensor 복사 · permute · .to(device)	D1-2
autograd	backward() 는 .grad 에 누적 → zero_grad → backward → step	D1-3
DataLoader	배치 (100, 1, 28, 28) · 600 iteration/epoch · CE 는 logits 입력	D1-4
학습 루프	train() 6 단계 · eval() + no_grad() · state_dict 저장 (MLP 3 epoch 약 0.9624)	D1-5
이미지	(H, W, C) uint8 BGR → (C, H, W) float32 RGB · resize 크기는 (W, H)	D1-6
영상	VideoCapture → read 루프 → VideoWriter · 30 FPS · 180 프레임	D1-7
종이 손글씨	naive 0.100 → 조명 보정 · Otsu · 무게중심 → 약 0.967 (domain shift)	D1-8

Day 2 로 가는 다리 — CNN 대표 구조

오늘 만든 학습 도구로, 더 어려운 데이터 CIFAR-10 에 LeNet · VGG · ResNet 을 학습시킵니다

Day 1 (오늘)		Day 2 (10/28 수)
`MLP 784-100-10 · MNIST`	→	MLP 로 CIFAR-10 이 안 되는 이유
`SimpleConvNet (Conv 1층)`	→	LeNet · AlexNet · VGG
`nn.Sequential · nn.Module`	→	깊이 문제와 BatchNorm
`Adam 2 epoch test 0.9830`	→	ResNet 구현 · CIFAR-10 85% 도전
`train_one_epoch · evaluate`	→	**그대로 사용**
전처리로 입력을 데이터 쪽으로	→	데이터 증강 (Day 2) · 전이학습 (Day 3)

MLP 의 한계 → LeNet · AlexNet · VGG → 깊이와 BatchNorm → ResNet → CIFAR-10 85%

오늘 만든 train_one_epoch · evaluate 를 그대로 사용합니다 — 노트북을 Drive 에 사본 저장해 두세요